

Relatório de Entrega 2

Projeto: Central de comandos por voz - Raspberry Pi 3 + kit Freenove

Versão do projeto: 0.2.0

Data: 2026

1. Introdução

Este projeto implementa uma central de comandos por voz offline para Raspberry Pi 3 com periféricos do kit Freenove. A solução reconhece comandos curtos em português, executa ações em LEDs, servomotor, LCD, sensor de distância e matriz de LEDs, e registra eventos localmente.

A entrega 2 consolida uma arquitetura modular em Python, com linha de comando, reconhecimento de fala, roteamento de intenções, manipulação de hardware, persistência em SQLite, testes automatizados e integração contínua.

2. Motivação

A principal motivação do projeto é mostrar uma solução de voz simples e demonstrável em hardware embarcado, sem depender de internet, APIs externas ou serviços em nuvem. Isso reduz pontos de falha e torna a apresentação mais previsível.

O uso de referência técnica foi importante para orientar a implementação e a montagem do sistema, especialmente nas partes de GPIO, I2C, áudio e reconhecimento de fala. Como material de apoio, o projeto se baseia em:

- documentação do Raspberry Pi;
- documentação do GPIO Zero;
- documentação do Vosk;
- documentação do smbus2;
- documentação própria do repositório, incluindo docs/ligacoes.md e docs/MIGRACAO.md.

3. Objetivo

Desenvolver uma central de comandos por voz local, executada no Raspberry Pi 3, capaz de:

- captar áudio de microfone USB;
- reconhecer comandos em português com Vosk;
- acionar periféricos do kit Freenove;
- mostrar feedback visual no LCD e na matriz de LEDs;
- registrar eventos e recados localmente;
- permitir teste em modo simulado.

4. Estrutura do repositório

A organização atual do repositório mostra a separação por responsabilidade:

```
centralvoz/  
- cli.py           interface de linha de comando  
- config.py        configuracao central  
- gpio_setup.py    escolha do backend de GPIO  
- doctor.py        diagnostico do ambiente  
- logging_setup.py configuracao de logs  
- utils.py         funcoes auxiliares  
- audio/           captura de audio
```

- speech/ motor Vosk
- commands/ intenções, roteador e handlers
- hardware/ drivers reais e simulados
- app/ controlador e loop principal
- storage/ SQLite para eventos e recados

tests/
examples/
docs/
.github/workflows/

Também existem exemplos executáveis numerados, documentação de montagem e migração, além de pipeline de CI.

5. Arquitetura implementada

O fluxo principal do sistema é:

1. o usuário inicia a fala por push-to-talk;
2. o microfone fornece blocos de áudio;
3. o motor Vosk transcreve a fala;
4. o roteador identifica a intenção;
5. o handler executa a ação;
6. o hardware recebe o comando;
7. o estado é exibido no LCD e registrado em SQLite.

A arquitetura separa claramente:

- **entrada de áudio** (audio/microphone.py);
- **reconhecimento** (speech/vosk_engine.py);
- **interpretação** (commands/router.py);
- **execução** (commands/handlers.py e app/controller.py);
- **hardware** (hardware/*);
- **persistência** (storage/db.py).

Essa divisão facilita testes, manutenção e troca de componentes sem alterar o fluxo inteiro.

6. Requisitos

6.1 Requisitos funcionais

ID	Requisito	Critério de verificação
RF01	Capturar áudio por microfone.	O sistema abre a entrada de áudio e processa blocos em streaming.
RF02	Reconhecer comandos de voz em português.	Uma frase cadastrada é transcrita e roteada para a intenção correta.
RF03	Operar em modo push-to-talk.	A captura começa ao ativar o gatilho e termina ao soltar.
RF04	Acionar LEDs conforme o comando reconhecido.	Comandos de ligar, desligar e piscar alteram o estado do LED.

ID	Requisito	Critério de verificação
RF05	Movimentar o servomotor para posições definidas.	Comandos de abrir, fechar e varrer executam ângulos previstos.
RF06	Ler o sensor de distância quando solicitado.	O sistema exibe a distância medida no LCD.
RF07	Exibir mensagens de estado no LCD.	O LCD apresenta resposta textual ao comando executado.
RF08	Registrar eventos e recados localmente.	Cada comando e cada ditado geram registro em SQLite.
RF09	Permitir modo ditado.	Ao ativar o ditado, o texto falado passa a ser salvo como nota.
RF10	Permitir consulta de recados e status.	Os comandos de listar recados, repetir, hora e status retornam dados válidos.

6.2 Requisitos não funcionais

ID	Requisito	Critério de verificação
RNF01	Executar em Raspberry Pi 3.	O projeto foi pensado para esse hardware e seus periféricos.
RNF02	Funcionar offline.	O reconhecimento usa Vosk local, sem API externa.
RNF03	Ter arquitetura modular.	Cada responsabilidade fica em um pacote próprio.
RNF04	Oferecer modo simulado.	--mock permite rodar sem GPIO, microfone ou modelo.
RNF05	Possuir testes automatizados.	pytest executa a suíte em ambiente simulado.
RNF06	Ter integração contínua.	O workflow do GitHub Actions valida testes e exemplos.
RNF07	Manter configuração centralizada.	config.toml, variáveis de ambiente e CLI seguem precedência definida.

7. Implementação da Entrega 2

7.1 Interface de linha de comando

A aplicação passou a ser operada pelo comando voz, com subcomandos úteis para diagnóstico e execução:

- voz doctor
- voz devices
- voz selftest
- voz text
- voz say "ligar led"

- voz run
- voz hello

Esse desenho reduz o custo de teste, facilita a demonstração e permite validar partes isoladas antes da integração completa.

7.2 Áudio e reconhecimento

A captura de áudio foi implementada em streaming, com negociação de taxa e suporte a microfones USB comuns. O motor de reconhecimento usa Vosk e pode operar com gramática restrita para melhorar a precisão dos comandos fixos.

O projeto também contempla o modo ditado, no qual o reconhecimento passa a ser livre e o texto é salvo localmente.

7.3 Comandos e handlers

A interpretação de texto foi separada do controle do hardware. O roteador converte frases reconhecidas em intenções, e cada intenção tem um handler próprio.

Isso permite:

- reduzir acoplamento;
- corrigir problemas de reconhecimento imperfeito;
- tratar negações corretamente;
- adicionar novos comandos sem alterar o controlador principal.

7.4 Hardware real e simulado

Os periféricos foram organizados em drivers reais e simulados. O repositório inclui:

- LED;
- servomotor;
- LCD I2C;
- sensor ultrassônico;
- matriz 8x8;
- gatilho por botão ou teclado.

O projeto também trata problemas comuns de Raspberry Pi, como escolha do backend de GPIO, uso correto de PWM e configuração de I2C.

7.5 Persistência e logs

Os eventos são gravados em SQLite em `logs/centralvoz.sqlite3`. O sistema também mantém registros em arquivo rotativo. Isso cobre o requisito de persistência local e ajuda na depuração.

8. Testes e integração contínua

O repositório possui suíte de testes em `tests/` cobrindo:

- roteamento de comandos;
- configuração;
- hardware simulado;
- utilitários;
- resolução do modelo Vosk;
- comportamento do modo mínimo.

A pipeline de CI executa os testes em Python 3.11, 3.12 e 3.13, verifica os exemplos do diretório `examples/` e valida a CLI em modo simulado.

9. Estado atual do projeto

A versão atual consolida a passagem de uma proposta inicial para uma implementação funcional. Em vez de depender apenas de documentação conceitual, o repositório agora contém:

- execução real via linha de comando;
- diagnóstico automático do ambiente;
- reconhecimento offline;
- ações em hardware;
- persistência local;
- testes e automação.

10. Conclusão

A Entrega 2 apresenta um projeto mais maduro, com arquitetura modular e foco em execução prática no Raspberry Pi 3. Os requisitos foram reescritos de forma testável, a motivação foi fortalecida com material de referência e a implementação passou a refletir o que realmente existe no repositório.

11. Referências

Documentação do projeto

- `README.md`
- `docs/ligacoes.md`
- `docs/MIGRACAO.md`

Referências externas

- Raspberry Pi Documentation: <https://www.raspberrypi.com/documentation/>
- GPIO Zero Documentation: <https://gpiozero.readthedocs.io/en/stable/>
- Vosk: <https://alphacephei.com/vosk/>
- Vosk installation: <https://alphacephei.com/vosk/install>
- smbus2 documentation: <https://smbus2.readthedocs.io/>